#중급반 4주차

이진 탐색 심화

<Binary Search - Advanced>

2p /<이진 탐색>
7p /<매개 변수 탐색>
18p /<LIS>
30p /<연습 문제>

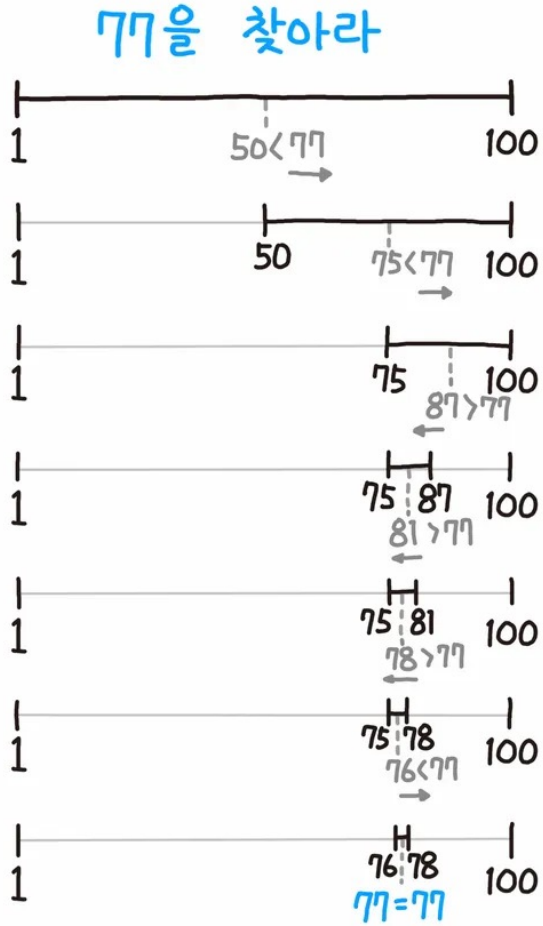
#이진 탐색

<Binary Search>

이진 탐색의 다양한 응용 방법에 대해 알아봅시다.

#이진 탐색

<Binary Search>



이진 탐색:

정렬된 데이터에서 특정 값을 갖는 원소를 빠르게 찾아내는 알고리즘

1. 현재 보고 있는 구간의 가운데를 찾고자 하는 값과 비교한다.
2. 비교 결과에 따라 구간을 왼쪽 절반 또는 오른쪽 절반으로 줄인다.

#이진 탐색

<Binary Search>

이진 탐색은 정렬된 데이터에서 매우 강력한 성능을 가집니다.

O(log N) 시간에 원하는 데이터를 찾을 수 있습니다.

“정렬되어 있다”를 다른 말로 단조성이라고 합니다.

어떤 데이터 A가 단조성을 가진다는 것은, $A_1 \leq A_2 \leq \cdots \leq A_N$ 또는 $A_1 \geq A_2 \geq \cdots \geq A_N$ 을 만족한다는 것을 의미합니다.

#이진 탐색

<Binary Search>

다음 글을 참고하면 이분 탐색을 쉽고 안전하게 구현할 수 있습니다.

https://web.archive.org/web/20230315044617/https://www.acmicpc.net/blog/view/109

#이진 탐색

<Binary Search>

이진 탐색은 단조성을 가지는 데이터에서만 사용 가능합니다.

따라서, 우리는 이진 탐색을 극한까지 응용하기 위해서,

단조성을 가지도록 문제를 변형시킬 것입니다.

무슨 말인지 자세히 알아봅시다.

#매개 변수 탐색

<Parametric Search>

매개 변수 탐색이란, 어떤 문제를 결정 문제로 바꾼 뒤, 이진 탐색으로 해결하는 기법입니다.

결정 문제란, 답이 “예” 또는 “아니오” 둘 중 하나인 문제입니다.
결정 문제의 예시는 다음과 같습니다.

- f(x): “x는 N 이상인가?”
- g(x): “x의 약수 개수는 k개 이상인가?”

#매개 변수 탐색

<Parametric Search>

2 나무꾼 미르코 (JUNGOL #5170)

<문제 설명>

- 나무 N그루가 있고, 각 나무의 높이 H_i가 주어진다.
- 절단기의 높이를 h로 설정하면,
  모든 나무에서 h보다 높은 부분만 잘라 가져간다.
- 총 M미터 이상의 나무를 얻을 수 있는 절단기 높이 h의 최댓값은?

<제약 조건>

- $1 \le N \le 1{,}000{,}000$
- $1 \le M \le 2{,}000{,}000{,}000$
- $0 \le H_i < 1{,}000{,}000{,}000$
- 나무 높이의 합은 M 이상, $h \ge 0$

<예시>

- N = 4, M = 7, 나무 높이 = [20, 15, 10, 17]
- h = 15 -> 5+0+0+2=7 미터를 얻으므로 가능함.
- h = 16 -> 4+0+0+1=5 미터를 얻으므로 불가능함.
- 따라서, 정답은 15

#매개 변수 탐색

<Parametric Search>

2 나무꾼 미르코 (JUNGOL #5170)

- 높이가 H_i인 나무에서 얻는 길이는 $\max(H_i - h, 0)$입니다.
  절단기보다 낮거나 같은 나무에서는 잘라 갈 부분이 없기 때문입니다.
- 따라서, 결정 문제 $F(h) = [\ \sum \max(H_i - h, 0) \geq M\ ]$ $(i = 1, \cdots, N)$ 로 변환할 수 있습니다.
- 중요: 단조성이 성립합니다.
  - h를 높이면 각 나무에서 얻는 길이는 줄거나 그대로입니다.
  - 따라서 한 높이에서 M미터를 채우지 못하면, 그보다 높은 곳에서도 불가능합니다.
  - 즉, F(h)를 모든 h에 대해 나열해 본다면, “예, 예, …, 예, 아니오, 아니오, …, 아니오”와 같이 생겼을 것입니다.
- 따라서, 이분 탐색을 진행할 수 있습니다.
- 중간 지점을 잡아서, 결정 문제의 답이 “예”라면 오른쪽 절반, “아니오”라면 왼쪽 절반으로 구간을 줄입니다.

- 시간복잡도는 $O(N \log(\max(H_i)+1))$입니다.

#매개 변수 탐색

<Parametric Search>

2 나무꾼 미르코 (JUNGOL #5170)

```cpp
#include <bits/stdc++.h>
using namespace std;

int N, M;
vector<int> H;

bool canCut(int height)
{
    long long sum = 0;

    for(int i=0; i < N; i++){
        int h = H[i];
        sum += max(h - height, 0);
    }
    return sum >= M;
}
```

```cpp
int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    cin >> N >> M;
    H.resize(N);
    for(int i = 0; i < N; i++) cin >> H[i];

    int lo = 0;
    int hi = *max_element(H.begin(), H.end());

    while (lo + 1 < hi){
        int mid = (lo + hi) / 2;
        if (canCut(mid)) lo = mid;
        else hi = mid;
    }

    cout << lo << '\n';
}
```

#매개 변수 탐색

<Parametric Search>

매개 변수 탐색은, 이진 탐색과 전혀 관계가 없어 보이는 문제에서도 등장합니다.

이런 문제에서는, 숨겨져 있는 단조성을 찾는 것이 중요합니다.

#매개 변수 탐색

<Parametric Search>

4 환경부의 나무 심기 프로젝트 (JUNGOL #5804)

<문제 설명>

- 직선 위 서로 다른 후보지 N곳에 나무 K그루를 심기
- 가장 가까운 두 나무 사이 거리가 최대가 되게 위치를 고르기

<제약 조건>

- $2 \leq K \leq N \leq 200{,}000$
- $0 \leq x_i \leq 10^9$

<예시>

- N = 5, K = 3, 후보지 위치 = [1, 2, 4, 8, 9]
- 위치 1, 4, 9에 나무를 심을 수 있고, 이 때 가장 가까운 두 나무 사이 거리는 3이 되어 최대

#매개 변수 탐색

<Parametric Search>

4 환경부의 나무 심기 프로젝트 (JUNGOL #5804)

- 후보지 위치 N개에서 K개를 고르는 경우의 수는 $\binom{N}{K}$개이므로, 모두 확인한다면 $O(N^K)$입니다.
  모든 경우를 확인하는 것은 효율적이지 않습니다.

- 그렇다면, 반대로 거리 d가 주어질 때, 나무를 몇 그루 심을 수 있는지 찾는 것은 어떨까요?

#매개 변수 탐색

<Parametric Search>

환경부의 나무 심기 프로젝트 (JUNGOL #5804)

- 위치가 [1, 2, 4, 8, 9]일 때를 생각해볼까요?

- d=3 일 때, 1에 심고 2는 건너뜁니다. 4와 8에 더 심으면 세 그루이므로 가능합니다.
- d=4 일 때, 1 다음에는 8에 심습니다. 더 이상 심을 수 없기에 두 그루만 심을 수 있습니다.

- 다른 위치를 골랐다면 d = 4에서도 세 그루를 심을 수 있었을까요?

#매개 변수 탐색

<Parametric Search>

4 환경부의 나무 심기 프로젝트 (JUNGOL #5804)

- 가장 왼쪽에 심어도 되는 이유는 다음과 같습니다.
  - 가능한 배치의 첫 나무를 가장 왼쪽 후보지로 옮겨 봅시다.
    이때, 다음 나무까지의 간격은 그대로이거나 더 커집니다.
  - 두 번째 나무부터도 같은 논리가 적용됩니다.
- 이 교체를 반복하면 가능한 배치를 왼쪽부터 고른 배치로 바꿀 수 있습니다.
- 따라서 왼쪽부터 심어도 K그루에 못 미치면 다른 배치도 불가능합니다.

#매개 변수 탐색

<Parametric Search>

환경부의 나무 심기 프로젝트 (JUNGOL #5804)

- 결정 문제 F(d) = “서로 d 이상 떨어지게 나무 K그루를 심을 수 있는가?” 를 생각할 수 있습니다.

- 어떤 배치가 d를 만족하면, 같은 배치는 d보다 작은 간격도 만족합니다. 따라서, 단조성이 성립합니다.
  즉, d를 늘릴 때 마지막으로 F(d)가 참인 지점이 정답입니다.
- 단조성이 성립하기에, 이분 탐색을 통해 답을 찾을 수 있습니다.

#매개 변수 탐색

<Parametric Search>

```cpp
#include <bits/stdc++.h>
using namespace std;

int N, K;
vector<int> X;

bool canPlant(int dist) {
    int cnt = 1;
    int last = X[0];

    for (int i = 1; i < N; i++) {
        if (X[i] - last >= dist){
            last = X[i];
            cnt++;
            if (cnt == K) return true;
        }
    }
    return false;
}
```

```cpp
int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    cin >> N >> K;
    X.resize(N);
    for(int i = 0; i < N; i++) cin >> X[i];

    sort(X.begin(), X.end());

    int lo = 1;
    int hi = X[N-1] - X[0] + 1;
    while (lo + 1 < hi) {
        int mid = (lo + hi) / 2;
        if (canPlant(mid)) lo = mid;
        else hi = mid;
    }

    cout << lo << '\n';
}
```

#가장 긴 증가하는 부분 수열 (LIS)

<LIS>

- 이진 탐색의 응용 중 하나로, 가장 긴 증가하는 부분 수열 문제(Longest Increasing Subsequence, LIS)를 O(N log N) 시간에 해결할 수 있습니다.

- 예시)
- 1 8 2 6 3 5 4 7
- 1 2 5는 증가하는 부분 수열입니다.
- 1 2 3 5 7은 증가하는 부분 수열 중 가장 깁니다. 따라서 LIS입니다.

#가장 긴 증가하는 부분 수열 (LIS)

<LIS>

- LIS는 다이나믹 프로그래밍을 활용해서 $O(N^2)$ 시간에 해결 가능합니다.

- 다음과 같이 DP 식을 정의합시다.
- DP[i] = i번째 수를 가장 마지막으로 하는 LIS의 길이
- DP[i] = i번째 수보다 작으며 먼저 등장하는 모든 j번째 수에 대하여, max(DP[j]) + 1

- 이진 탐색을 활용해 시간복잡도를 개선해 봅시다.

#가장 긴 증가하는 부분 수열 (LIS)

<LIS>

- 불필요한 정보를 삭제해 시간복잡도를 개선합니다.
  - 다음과 같은 수열을 생각해 봅시다: 3 5 1 2 4
  - 수열의 4번째 수까지 봐서, DP 식을 완성했다고 합시다.
  - DP[1] = 1 DP[2] = 2 DP[3] = 1 DP[4] = 2

  - 사실, DP[2] = 2라는 정보는 쓸모 없는 정보입니다. 왜냐하면, DP[4]가 DP[2]의 상위 호환 정보이기 때문입니다.
  - DP[2]는 2번째로 끝나는 수를 마지막으로 하는 LIS이므로, 3 5를 나타냅니다.
  - DP[4]는 4번째로 끝나는 수를 마지막으로 하는 LIS이므로, 1 2를 나타냅니다.

  - 이 뒤에 마지막 수 4를 추가로 붙이면서, 수열이 여전히 증가하도록 하려면, 더 낮은 수로 끝나는 1 2를 사용해야 합니다.

#가장 긴 증가하는 부분 수열 (LIS)

<LIS>

- 따라서, 우리는 다음과 같이 배열 L을 관리할 것입니다.
  - L[i] = 증가하는 부분 수열의 길이가 i가 되도록 하는 마지막 수의 값 중 가장 낮은 값

- 배열 L은 단조성을 만족합니다.
  - 증가하는 부분 수열의 길이가 더 길어진다면, 마지막 수의 값 역시 커집니다.
  - 잘 생각해보면 당연한 결과입니다. 길이를 늘리려면, 뒤에 마지막 수보다 큰 수를 넣어야 하기 때문입니다.

#가장 긴 증가하는 부분 수열 (LIS)

<LIS>

- 배열 L이 변하는 과정을 예시를 통해 살펴봅시다.

3 5 1 2 4

- 배열 L (L[i] = 증가하는 부분 수열의 길이가 i가 되도록 하는 마지막 수의 값 중 가장 낮은 값)
- 편의상, 배열의 인덱스 0은 생략합니다.

i	1	2	3	4	5	6
L[i]	-	-	-	-	-	-

#가장 긴 증가하는 부분 수열 (LIS)

<LIS>

- 배열 L이 변하는 과정을 예시를 통해 살펴봅시다.

3 5 1 2 4

- 배열 L (L[i] = 증가하는 부분 수열의 길이가 i가 되도록 하는 마지막 수의 값 중 가장 낮은 값)

i	1	2	3	4	5	6
L[i]	3	-	-	-	-	-

#가장 긴 증가하는 부분 수열 (LIS)

<LIS>

- 배열 L이 변하는 과정을 예시를 통해 살펴봅시다.

3 5 1 2 4

- 배열 L (L[i] = 증가하는 부분 수열의 길이가 i가 되도록 하는 마지막 수의 값 중 가장 낮은 값)

i	1	2	3	4	5	6
L[i]	3	5	-	-	-	-

#가장 긴 증가하는 부분 수열 (LIS)

<LIS>

- 배열 L이 변하는 과정을 예시를 통해 살펴봅시다.

3 5 1 2 4

- 배열 L (L[i] = 증가하는 부분 수열의 길이가 i가 되도록 하는 마지막 수의 값 중 가장 낮은 값)

i	1	2	3	4	5	6
L[i]	1	5	-	-	-	-

#가장 긴 증가하는 부분 수열 (LIS)

<LIS>

- 배열 L이 변하는 과정을 예시를 통해 살펴봅시다.

3 5 1 2 4

- 배열 L (L[i] = 증가하는 부분 수열의 길이가 i가 되도록 하는 마지막 수의 값 중 가장 낮은 값)

i	1	2	3	4	5	6
L[i]	1	2	-	-	-	-

#가장 긴 증가하는 부분 수열 (LIS)

<LIS>

- 배열 L이 변하는 과정을 예시를 통해 살펴봅시다.

3 5 1 2 4

- 배열 L (L[i] = 증가하는 부분 수열의 길이가 i가 되도록 하는 마지막 수의 값 중 가장 낮은 값)

i	1	2	3	4	5	6
L[i]	1	2	4	-	-	-

#가장 긴 증가하는 부분 수열 (LIS)

<LIS>

- 수 X를 추가하려고 한다고 합시다.
  - “증가하는 부분 수열”이려면, X보다 작은 수 뒤에 추가해야 합니다.
  - “가장 긴”이려면, i가 가장 커야 합니다.
  - 따라서, L[i]가 X보다 작으면서, i가 가장 큰 위치를 찾으면 됩니다.

- 배열 L이 단조성을 가지기 때문에, 이 위치 i를 O(log N)에 찾을 수 있습니다.
- 이후, L[i+1]의 값을 X로 교체하면 됩니다.
- 사실 i+1은 lower_bound(L.begin(), L.end(), X)와도 같습니다. 이를 사용하면 편하게 구현할 수 있습니다.

- 배열 L (L[i] = 증가하는 부분 수열의 길이가 i가 되도록 하는 마지막 수의 값 중 가장 낮은 값)

i	1	2	3	4	5	6
L[i]	1	5	-	-	-	-

#가장 긴 증가하는 부분 수열 (LIS)

<LIS>

1 최장 증가 부분 수열 (JUNGOL #4529)

```cpp
#include <bits/stdc++.h>
using namespace std;

const int INF = 2'100'000'000;

int main()
{
    int N; cin >> N;
    vector<int> A(N+1), L(N+1);
    for (int i = 1; i <= N; i++) {
        cin >> A[i];
        L[i] = INF;
    }
    for (int i = 1; i <= N; i++) {
        int pos = lower_bound(L.begin(), L.end(), A[i]) - L.begin();
        L[pos] = A[i];
    }
    int len = 0;
    for (int i = 1; i <= N; i++){
        if (L[i] < INF) len = i;
    }
    cout << len;
}
```

#예제

2 나무꾼 미르코 (JUNGOL #5170)

매개 변수 탐색 문제입니다.

4 환경부의 나무 심기 프로젝트 (JUNGOL #5804)

주어진 문제 상황에서 단조성을 찾기

1 최장 증가 부분 수열 (JUNGOL #4529)

매개 변수 탐색 문제입니다.

#연습 문제

4 사냥꾼 (JUNGOL #2634)

PPT에서 해설한 문제입니다.

3 숫자구슬 (JUNGOL #5461)

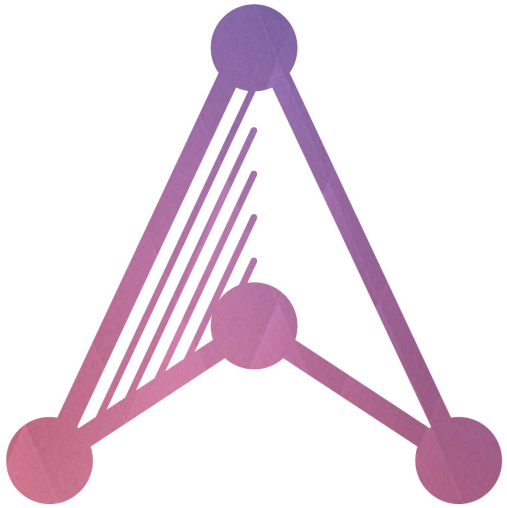
LIS는 자주 사용되는 알고리즘이니 꼭 알아둬야 합니다.

5 책 복사하기 (JUNGOL #1137)

PPT에서 해설한 “K번째 수”와 비슷한 문제입니다.

4 전깃줄(중) (JUNGOL #1257)

결정 문제 f(X): “평균이 X일 때, 가치를 K 이상으로 만들 수 있는가?”
종이에 평균의 계산식을 써놓고 잘 정리해 봅시다.
다이나믹 프로그래밍에 대한 깊은 이해가 필요합니다.



A L O H A

The algorithm club.